

OPEN SPECIFICATION

PRD & ARCHITECTURE · V0.1

Loomwork

An open operating system for portable personal agents.

Defines two open specifications — the **Agent Container Image (ACI)**, a signed, portable format that lets agents run identically across IDEs, devices, and runtimes; and the **Agent Mesh Protocol (AMP)**, a peer-to-peer wire protocol for agent-to-agent delegation, capability negotiation, and credit accounting. Together they close the gap left by MCP and turn agents into first-class, composable, ownable units of computing.

Document Loomwork PRD & Architecture

Version 0.1 · Draft

Date 28 September 2026

Status Open for community review

Scope ACI spec · AMP wire format · market thesis

Table of Contents

- 1. Executive Summary 3**
- 2. Market Thesis 4**
 - 2.1 What is trending on GitHub 4
 - 2.2 The seven drivers of traction 4
 - 2.3 The gap nobody has claimed 5
 - 2.4 Why now 5
- 3. Loomwork Architecture Overview 6**
 - 3.1 Layer responsibilities 6
 - 3.2 Design principles 7
 - 3.3 What is not in scope 7
- 4. ACI — Agent Container Image Specification 8**
 - 4.1 Goals and non-goals 8
 - 4.2 File layout 8
 - 4.3 Manifest schema 9
 - 4.4 Persona block 10
 - 4.5 Skills graph 10
 - 4.6 Tool bindings 11
 - 4.7 Sandbox spec 12
 - 4.8 Signing and provenance 12
 - 4.9 Runtime contract 12
 - 4.10 Versioning and compatibility 13
 - 4.11 Conformance requirements 13
- 5. AMP — Agent Mesh Protocol Wire Format 15**
 - 5.1 Design goals 15
 - 5.2 Transport bindings 15
 - 5.3 Message envelope 15
 - 5.4 Capability negotiation handshake 16
 - 5.5 Core RPCs 18

5.5.1 amp/delegate 18

5.5.2 amp/negotiate 19

5.5.3 amp/report 19

5.5.4 amp/credit 20

5.5.5 amp/subscribe and amp/cancel 20

5.6 Task graph and shared state 20

5.7 Credit and accounting 21

5.8 Security model 21

5.9 Error code registry 21

5.10 Version negotiation and backwards compatibility 22

6. Relationship to Existing Protocols 23

7. Roadmap and Risks 24

7.1 Roadmap 24

7.2 Top risks and mitigations 24

7.3 Open questions 25

Appendix A. Glossary 26

1. Executive Summary

The dominant AI agents of 2026 — OpenClaw, OpenHands, GitHub Copilot Mission Control, Bolt.new, Totalum — share a common architectural shape: a capable LLM, a sandbox, a set of tools exposed via the Model Context Protocol (MCP), and an IDE- or chat-bound surface. They are powerful, but they are also sealed. The agent's persona, skills, memory schema, and tool bindings live inside one vendor's runtime. You cannot take an agent you built in Cursor and run it in Claude Code. You cannot let your laptop agent talk to your phone agent. Your memory does not follow you across vendors. And the apps that vibe-coded tools produce ship with no verification gate, which is why serious teams still hand-review every line.

Loomwork is an open-source operating system for portable personal agents that closes these gaps. It is built on two specifications introduced by this document. The **Agent Container Image (ACI)** is a signed, portable package format for an agent — persona, skill graph, MCP tool bindings, sandbox policy, memory schema, and Sigstore provenance — that runs identically inside any compliant runtime. The **Agent Mesh Protocol (AMP)** is a peer-to-peer wire protocol, layered on top of MCP, that lets agents delegate, negotiate capabilities, exchange credit, and subscribe to each other's events. Together they turn an agent from a vendor-bound artifact into a first-class, ownable, composable unit of computing — the way Docker turned a server process into a portable artifact, and the way HTTP turned a document into a network resource.

This document specifies ACI and AMP at reference-grade depth, suitable for independent implementation. It opens with a market thesis grounding the design in concrete GitHub trends from the last 90 days, then presents the four-pillar architecture, then dedicates one chapter each to ACI and AMP, including JSON schemas, wire-format examples, sequence diagrams, error-code registries, and conformance requirements expressed in RFC 2119 language. The document closes with a comparison to existing protocols, a 90-day to 24-month roadmap, and an explicit list of open questions. The intended audience is founding engineers who will build the reference runtime and seed the first agents, and investors evaluating whether the timing is right to back an open agent standard.

The ask

Ship ACI v0.1 and AMP v0.1 as open specifications within 90 days, accompanied by a reference Rust runtime, three example agents packaged as ACIs, and a conformance test suite. Apache 2.0 for code, CC-BY 4.0 for the specifications. Working group governed under a neutral foundation (candidate: OpenSSF or Linux Foundation AI & Data).

2. Market Thesis

Every design choice in Loomwork is grounded in observed developer behavior on GitHub over the last 60 to 90 days. This section enumerates what is trending, extracts the underlying drivers of that traction, and identifies the specific gap Loomwork fills.

2.1 What is trending on GitHub

The most striking trend of late 2025 and early 2026 is the velocity of **OpenClaw**, a Node.js framework that provides OS-level control to AI agents and ships 20+ messaging-platform adapters. Star trackers widely report it went from zero to roughly 390,000 stars in approximately 60 days, surpassing totals that took React 13 years to reach. DigitalOcean noted a 9,000 to 60,000-star jump in a single 72-hour window in late January 2026. Whatever one thinks of the project itself, the velocity is unprecedented and signals that a large latent population of developers was waiting for exactly this category of tool.

Alongside OpenClaw, **OpenHands** (formerly OpenDevin) has stabilized as the reference open-source autonomous coding agent — a Docker-sandboxed, MIT-licensed platform cited by essentially every 'best of 2026' list. The {b('Model Context Protocol')} has become the de-facto standard for connecting agents to tools: Anthropic's reference server repository and hundreds of community MCP servers (Context7, GitHub, Playwright, Filesystem, Totalum, Twitter) now anchor the trending feed. {b('Ollama')} and {b('vLLM')} continue to grow as the local-first inference substrate, and {b('Bolt.new')} and {b('Totalum')} have popularized vibe-coded app generation — natural-language input to deployable Next.js application. Most recently, GitHub itself exposed {b('Copilot Mission Control')}, an async cloud coding agent that fans out GitHub issues to background agents that open PRs.

2.2 The seven drivers of traction

Reading these projects against their traction, seven recurring drivers emerge. Each driver is independent, and the most successful projects hit several at once.

Driver	What it looks like in practice
1. Open-source and self-hostable	Developers are tired of per-seat SaaS pricing. OpenHands, Ollama, and the MCP ecosystem let you own the runtime.
2. Protocol-driven composability	MCP won because one server works in Claude Code, Cursor, Codex, Copilot, and Windsurf. Write-once, run-in-five-clients is the TCP/IP moment for agents.
3. OS-level reach	OpenClaw's killer feature isn't the LLM — it's that one agent drives Discord, Telegram, WhatsApp, Slack, X, the filesystem, and the browser. Reach beats chat.
4. Async and parallelizable	Copilot Mission Control and OpenHands both let you fan out 10 issues to 10 agents and review PRs later. The mental model shifted from pair-programmer to spawnable junior dev team.
5. Local-first and private	Ollama's continued growth and OpenClaw's 'runs on your machine' framing reflect a real constituency that will not send data to OpenAI.

Driver	What it looks like in practice
6. Vibe-coded app generation	Bolt.new + Totalum lowered the floor. The audience expanded from developers to anyone who can describe an app.
7. Viral launch craft	OpenClaw's 9K-to-60K-stars-in-72-hours was driven by a great demo gif, a clean README, and a synchronized Hacker News + Reddit + X drop.

Loomwork is engineered to hit drivers 1 through 5 by design, and to amplify 6 and 7 via the Vibe-to-Verified pipeline and the ACI marketplace respectively. No existing trending project hits all seven.

2.3 The gap nobody has claimed

Look at what is missing from the trending list. MCP standardizes how an agent calls a tool, but the agent itself — its persona, skills, memory schema, prompts, sandbox config, and signing identity — is still locked inside Cursor, Claude Code, Copilot, or Codex. You cannot `docker pull` an agent. There is no agent-to-agent protocol: MCP is agent-to-tool only, so your research agent cannot formally delegate a subtask to your coding agent. There is no continuous personal memory that follows the user across vendors; every session starts fresh, and your Claude in Cursor does not know what your Copilot in the browser did yesterday. Vibe-coded apps ship broken because there is no open, end-to-end verification gate. And there is no personal agent mesh — your laptop agent, phone agent, and home-server agent do not coordinate without a central cloud.

2.4 Why now

MCP was introduced by Anthropic in late 2024 and reached mainstream adoption through 2025. By mid-2026 it is the lingua franca of agent-to-tool communication. The natural next layer is the agent itself as a portable unit. Whoever defines that layer in 2026 — with an open, well-specced, community-governed format — will set the standard for the next decade of agent tooling, the same way Docker's OCI image format set the standard for cloud-native deployment after Linux containers had been around for years without a common packaging layer. The window is narrow: OpenClaw's viral trajectory shows the demand exists, and a closed-vendor capture of the agent-portability layer is the main competitive risk. The time to ship an open spec is now.

3. Loomwork Architecture Overview

Loomwork is best understood as a five-layer stack. The two layers Loomwork specifies — ACI and AMP — sit in the middle, above the existing MCP and LLM-runtime substrate and below the application and mesh-runtime layers. Each layer is independently replaceable, and every layer above the ACI/AMP core is optional: a single-agent single-device setup uses only ACI + MCP, while a full multi-device team deployment uses all five layers.

Loomwork System Architecture

Five layers from runtime substrate to user-facing applications. AMP and ACI are the open specs; the rest are existing components Loomwork composes.

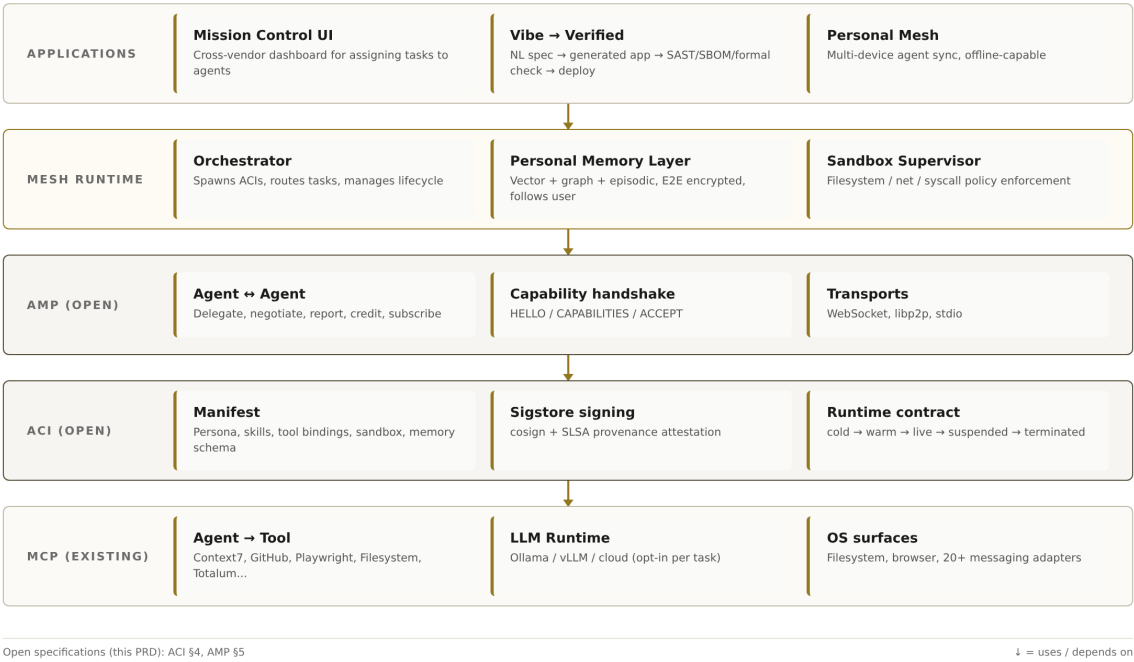


Figure 1. Loomwork five-layer architecture. ACI and AMP are the open specifications; the other layers compose existing components.

3.1 Layer responsibilities

Layer	Owner	Responsibility
Applications	Vendor / user	Mission Control UI, Vibe-to-Verified pipeline, Personal Mesh sync UI. Talks to Mesh Runtime only.
Mesh Runtime	Loomwork runtime	Spawns ACIs, routes AMP traffic, manages lifecycle, enforces sandbox policy, hosts Personal Memory Layer. The orchestrator.
AMP	Loomwork spec (this document)	Peer-to-peer JSON-RPC wire protocol between agents. Delegate, negotiate, report, credit, subscribe.
ACI	Loomwork spec (this document)	Signed, portable agent image: manifest + persona + skills + tool bindings + sandbox + memory schema + provenance.
MCP + LLM runtime + OS	Existing open source	Agent-to-tool protocol (MCP), inference (Ollama / vLLM / cloud), OS surfaces (filesystem, browser, messaging).

3.2 Design principles

Four principles constrain every design decision in this document. First, **open by default**: every wire format, file format, and protocol described here is licensed to allow independent implementation without permission or royalty. Second, **local-first**: the reference runtime must work end-to-end on a single laptop with no network dependency; cloud and remote peers are opt-in accelerators, not requirements. Third, **composable, not monolithic**: an agent is a unit, but it is also a node in a graph; delegation, negotiation, and credit are first-class operations. Fourth, **verified before trusted**: every artifact an agent produces — code, file, network request, memory write — carries a signed provenance attestation so that downstream consumers can audit who did what, when, with which capability.

3.3 What is not in scope

Loomwork deliberately does not standardize the LLM runtime, the sandbox implementation, or the UI. Those are intentionally left to existing solutions (Ollama, vLLM, Docker, Firecracker, vendor IDEs). Loomwork also does not standardize a marketplace, payment rails, or governance — those are expected to emerge as separate layers above the spec, and the spec is designed to be marketplace-agnostic. Finally, Loomwork does not define a 'safe agent' — it defines the contract under which an agent's safety properties can be verified, but the verification itself is the responsibility of the Vibe-to-Verified pipeline, which is a runtime component, not a spec component.

4. ACI — Agent Container Image Specification

The Agent Container Image (ACI) is a signed, portable package format for an AI agent. An ACI file bundles everything required to reproduce an agent's behavior across compliant runtimes: persona, skill graph, MCP tool bindings, sandbox policy, memory schema, and cryptographic provenance. The design goals are portability (an ACI built for Cursor runs in Claude Code, on a phone, or on a Raspberry Pi), reproducibility (the same ACI + the same inputs produces the same outputs, modulo model nondeterminism), and auditability (every action is traceable to a signed identity).

4.1 Goals and non-goals

ACI's primary goal is to make an agent a portable artifact, analogous to how the OCI Image Format made a server process a portable artifact. An ACI must be self-describing: a runtime that has never seen the ACI before must be able to load it, verify its signature, prepare its sandbox, expose its declared tool bindings, and start it producing output — all without manual configuration. Secondary goals include minimal size (a basic ACI should fit in under 100 KB excluding bundled model weights), strong provenance (Sigstore + SLSA), and version independence (ACIs from v0.1 must remain loadable by v1.0 runtimes, with explicit capability negotiation for newly introduced features).

Explicit non-goals: ACI does not standardize the inference engine, the sandbox implementation, or the prompt template syntax. Prompt templating is delegated to the persona block, which may use any text format the runtime understands (Jinja, Mustache, or plain text with variable interpolation). Sandboxing is specified as policy intent (filesystem paths, network egress rules, syscall allowlist), not as implementation; the runtime is free to use Docker, Firecracker, gVisor, or native Linux seccomp as long as the declared policy is enforced.

4.2 File layout

An ACI is a tar archive (gzip-compressed by default) with the .aci extension. The top-level layout is fixed; runtimes MUST reject archives that omit any required file.

```
# Top-level layout of an ACI archive
my-agent-v1.2.3.aci
├─ manifest.json # Required. Schema in §4.3.
├─ persona/
│   └─ system_prompt.md # Required. The agent's identity.
│       └─ few_shot.jsonl # Optional. Few-shot exemplars.
├─ skills/
│   └─ graph.json # Required. DAG of named capabilities (§4.5).
│       └─ src/ # Optional. Skill implementation source.
├─ tools/
│   └─ bindings.json # Required. MCP server refs + capability URLs (§4.6).
├─ memory-schema.json # Required. Shape of agent's persistent memory (§5 of Memory spec).
├─ sandbox.json # Required. Sandbox policy (§4.7).
├─ signatures/
│   └─ manifest.sig # Required. Sigstore cosign signature over manifest.json.
│       └─ manifest.cert # Required. Signing certificate (Fulcio-issued).
│           └─ slsa.intoto.json # Required. SLSA Level ≥3 provenance attestation.
└─ README.md # Optional. Human-readable description.
```

4.3 Manifest schema

The manifest is the ACI's root of trust. It is a JSON document with a strict schema; runtimes **MUST** reject manifests that fail validation. Every other file in the archive is referenced by digest from the manifest, so the signature over the manifest transitively covers the entire ACI.

```
// manifest.json — JSON Schema (Draft 2020-12)
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://loomwork.dev/schemas/aci/manifest-v0.1.json",
  "title": "ACI Manifest v0.1",
  "type": "object",
  "required": ["apiVersion", "kind", "metadata", "persona",
    "skills", "tools", "memory", "sandbox", "digests"],
  "properties": {
    "apiVersion": { "const": "aci.loomwork.dev/v0.1" },
    "kind": { "const": "Agent" },
    "metadata": {
      "type": "object",
      "required": ["name", "version", "architecture", "os"],
      "properties": {
        "name": { "type": "string", "pattern": "^[a-z0-9][a-z0-9-]*$" },
        "version": { "type": "string", "pattern": "^v?\\d+\\.\\d+\\.\\d+$" },
        "architecture": { "enum": ["amd64", "arm64", "wasm"] },
        "os": { "enum": ["linux", "darwin", "windows", "any"] },
        "description": { "type": "string" },
        "license": { "type": "string", "default": "Apache-2.0" },
        "homepage": { "type": "string", "format": "uri" }
      }
    },
    "persona": { "$ref": "#/$defs/PersonaRef" },
    "skills": { "$ref": "#/$defs/SkillsRef" },
    "tools": { "$ref": "#/$defs/ToolsRef" },
    "memory": { "$ref": "#/$defs/MemoryRef" },
    "sandbox": { "$ref": "#/$defs/SandboxRef" },
    "digests": {
      "type": "object",
      "description": "SHA-256 digest of every other file in the archive.",
      "additionalProperties": { "type": "string", "pattern": "^sha256:[0-9a-f]{64}$" }
    }
  }
}
```

The **digests** map is the integrity anchor. Every key is a path relative to the archive root; every value is the SHA-256 of that file's bytes. The signature in **signatures/manifest.sig** is computed over the canonical JSON serialization of **manifest.json** (RFC 8785 JCS). A runtime verifies an ACI by recomputing every digest, checking the manifest signature against the supplied certificate, and confirming the certificate chains to a trusted Fulcio root.

4.4 Persona block

The persona block binds the agent's identity, behavioral contract, and model preferences. It is intentionally flexible: the system prompt is plain text, and model preferences are hints, not requirements — the runtime is free to substitute any model that satisfies the declared capability floor.

```
// persona block (excerpt from manifest.json)
"persona": {
  "systemPrompt": "persona/system_prompt.md",
  "fewShot": "persona/few_shot.jsonl", // optional
  "modelPrefs": {
    "minContextWindow": 32768,
    "preferred": ["claude-sonnet-4.5", "gpt-5", "llama-3.3-70b"],
    "fallback": ["llama-3.1-8b"],
    "temperature": 0.2,
    "tokenBudget": {
      "perTurn": 4096,
      "perSession": 131072,
      "hardLimit": 1048576
    }
  },
  "capabilities": {
    "supportsStreaming": true,
    "supportsTools": true,
    "supportsVision": false,
    "supportsCodeExecution": true
  }
}
```

The `tokenBudget` field is the runtime's enforcement hook. When `hardLimit` is reached, the runtime **MUST** refuse to dispatch the next LLM call and **MUST** emit an `on_budget_exhausted` event. This prevents a runaway agent from incurring unbounded cost, which is the single most common operational failure mode in production agent deployments.

4.5 Skills graph

Skills are the agent's named capabilities — a directed acyclic graph (DAG) where each node is a skill with an input contract, an output contract, and an optional implementation reference. A skill without an implementation is a pure prompt-level capability; a skill with an implementation is backed by executable code (Python, JS, or Wasm) that runs in the sandbox. The runtime resolves skill invocations by walking the DAG, checking that the caller's capability token covers the requested skill, and dispatching either to the LLM or to the implementation.

```
// skills/graph.json – DAG of named capabilities
{
  "skills": [
    {
      "name": "search_web",
      "description": "Issue a web search query and return ranked results.",
      "inputs": { "query": "string", "maxResults": "int? = 5" },
      "outputs": { "results": "array<SearchResult>" },
      "requires": [],
      "impl": null
    },
    {
      "name": "summarize_page",
      "description": "Fetch a URL and return a 3-paragraph summary.",
      "inputs": { "url": "string" },
      "outputs": { "summary": "string" },
      "requires": ["search_web"],
      "impl": { "type": "wasm", "entry": "skills/src/summarize.wasm" }
    }
  ]
}
```

4.6 Tool bindings

Tool bindings are the bridge between the ACI and the MCP ecosystem. Each binding references an MCP server (by URL or by local-socket path), declares which capabilities the agent is allowed to invoke on that server, and binds a capability token scope. The runtime enforces these bindings: an attempt to call an undeclared tool **MUST** be denied and logged.

```
// tools/bindings.json
{
  "bindings": [
    {
      "name": "filesystem",
      "mcpServer": "stdio:///usr/local/bin/mcp-fs",
      "allowedTools": ["read_file", "write_file", "list_dir"],
      "scope": { "paths": ["$HOME/agent-workspace/**"] }
    },
    {
      "name": "browser",
      "mcpServer": "ws://localhost:8765",
      "allowedTools": ["navigate", "click", "extract_text"],
      "scope": { "domains": ["*.wikipedia.org", "arxiv.org"] }
    }
  ]
}
```

4.7 Sandbox spec

The sandbox spec declares the agent's filesystem, network, and syscall rights. The runtime **MUST** enforce every clause; failure to enforce is a critical conformance violation. The spec is intentionally implementation-agnostic: a runtime may use Docker, Firecracker, gVisor, or native seccomp, provided the declared policy is honored.

Clause	Type	Example	Default if omitted
fs.read	array	['\$HOME/docs/**', '/tmp/agent/**']	[] (no reads)
fs.write	array	['/tmp/agent/**']	[] (no writes)
net.egress	array	['*.wikipedia.org', 'api.github.com']	[] (no egress)
net.listen	array	[]	[] (no listening)
syscalls.allow	array	['read', 'write', 'openat', 'close']	runtime default
syscalls.deny	array	['execve', 'fork']	runtime default
resources.cpu	string (cores)	2	1
resources.memory	string (bytes)	4GiB	1GiB
resources.disk	string (bytes)	10GiB	1GiB
time.maxWall	string (duration)	30m	10m

4.8 Signing and provenance

Every ACI **MUST** be signed with a Sigstore cosign signature over the canonical-JSON serialization of `manifest.json`. The signing certificate **MUST** be issued by a Fulcio certificate authority (the public Sigstore CA is acceptable; private CAs are permitted for internal deployments). The signature and certificate are stored in `signatures/manifest.sig` and `signatures/manifest.cert` respectively. An ACI without a valid signature **MUST** be rejected by the runtime; an ACI with a signature but no SLSA attestation **MUST** be flagged as unverified-provenance and loaded only if the user explicitly opts in.

The SLSA provenance attestation (`signatures/slsa.intoto.json`) follows the in-toto statement format and **MUST** meet at least SLSA Build Level 3. The attestation captures the build environment, the source commit hash, the build steps, and the materials consumed. This is what enables downstream consumers to answer the question 'was this agent built from the source I think it was built from?' — which is the minimum bar for trusting an agent that may run with filesystem and network access.

4.9 Runtime contract

An ACI moves through five lifecycle states. Every transition emits a signed event that subscribers (the mesh runtime, the UI, telemetry) can react to. The state machine is intentionally minimal — five states is enough to express every meaningful operational behavior without over-constraining runtime implementations.

ACI Lifecycle State Machine

An ACI moves through five lifecycle states. Events are emitted on every transition; subscribers (mesh, UI, telemetry) react to them.

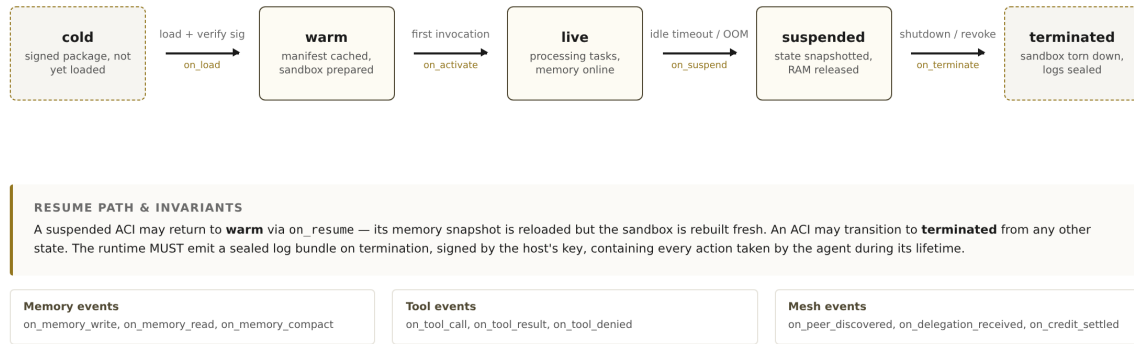


Figure 2. ACI lifecycle state machine. Terminal states (cold, terminated) are dashed. Resume path returns a suspended ACI to warm, not live.

4.10 Versioning and compatibility

ACI follows semantic versioning at the manifest level. The `apiVersion` field uses Kubernetes-style group-version syntax (e.g. `aci.loomwork.dev/v0.1`). Within a major version, runtimes **MUST** accept any minor or patch version. Across major versions, runtime **MUST** negotiate via the `apiVersion` field and **MUST** refuse to load ACIs whose major version exceeds the runtime's supported maximum. A `v0.x` runtime **MAY** load `v0.y` ACIs where $y < x$ by treating unrecognized fields as warnings, not errors. Once the spec reaches `v1.0`, forward compatibility is guaranteed for two years.

4.11 Conformance requirements

The keywords **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this section are to be interpreted as described in RFC 2119.

#	Requirement	Level
C1	Runtime MUST verify manifest signature before loading ACI.	MUST
C2	Runtime MUST verify all file digests against the manifest's digests map.	MUST
C3	Runtime MUST reject ACI whose manifest fails JSON Schema validation.	MUST
C4	Runtime MUST enforce every clause of <code>sandbox.json</code> .	MUST
C5	Runtime MUST emit a signed event on every lifecycle transition.	MUST
C6	Runtime MUST refuse LLM calls that exceed <code>persona.tokenBudget.hardLimit</code> .	MUST
C7	Runtime MUST refuse tool calls not declared in <code>tools/bindings.json</code> .	MUST
C8	Runtime SHOULD cache warm ACIs for at least 5 minutes after last activity.	SHOULD
C9	Runtime SHOULD persist a sealed log bundle on termination.	SHOULD
C10	Runtime MAY support ACIs without SLSA attestation if user opts in.	MAY

#	Requirement	Level
C1 1	Runtime MAY support multiple ACI versions concurrently.	MAY

5. AMP — Agent Mesh Protocol Wire Format

The Agent Mesh Protocol (AMP) is a peer-to-peer wire protocol that lets agents communicate with each other. Where MCP standardizes how an agent calls a tool, AMP standardizes how an agent delegates a task to another agent, negotiates capabilities, reports progress, and exchanges credit for work performed. AMP is layered strictly on top of MCP at the conceptual level — an agent that speaks AMP also speaks MCP — but the two protocols use independent transports and message namespaces. AMP is designed to be asynchronous, capability-bounded, credit-accountable, and transport-agnostic.

5.1 Design goals

AMP's primary design goal is to make agent-to-agent interaction as routable, observable, and accountable as agent-to-tool interaction already is under MCP. Four sub-goals follow. First, the protocol must be peer-to-peer: there is no central broker, and any two compliant runtimes must be able to converse directly. Second, every interaction must be capability-bounded: an agent cannot invoke another agent's skill unless the callee has explicitly granted a capability token for that skill. Third, the protocol must be credit-accountable: every unit of work exchanged between agents is settled by a signed credit receipt, allowing offline reconciliation and preventing freeloading. Fourth, the protocol must be transport-agnostic: the same logical message can be carried over WebSocket, libp2p, or stdio without changing its semantics.

5.2 Transport bindings

AMP defines three transport bindings in v0.1. Implementations **MUST** support at least one; **SHOULD** support two; **MAY** support all three. The choice of transport does not affect message semantics, only the framing.

Transport	Use case	Framing	Discovery
WebSocket	Local network, same-machine agents, IDE↔runtime	JSON-RPC 2.0 over WS frames	Well-known port 7878 + mDNS
libp2p	Cross-device, internet-scale, NAT traversal	JSON-RPC 2.0 inside libp2p streams	DHT + PeerId advertise
stdio	Parent process ↔ child agent (e.g. CLI tool)	Newline-delimited JSON-RPC 2.0	Spawned with --amp-stdio flag

5.3 Message envelope

AMP messages are JSON-RPC 2.0 objects with three AMP-specific extensions. The **amp** field carries protocol metadata; the **capabilities** field lists the capability tokens the sender is presenting; the **provenance** field traces the request back to its originating ACI. Runtimes **MUST** reject any message missing these fields when the message is an AMP-namespaced method.


```
// AMP message envelope – JSON-RPC 2.0 + extensions
{
  "jsonrpc": "2.0",
  "id": "req_01H8...", // string, ULID
  "method": "amp/delegate", // namespace: amp/*
  "params": { /* method-specific */ },
  "amp": {
    "version": "0.1",
    "traceId": "trace_01H8...", // propagates across delegations
    "sessionId": "sess_01H8...", // established in HELLO/ACCEPT
    "ttl": 30 // seconds; runtime drops after
  },
  "capabilities": [ // signed capability tokens
    {
      "id": "cap_01H8...",
      "skill": "search_web",
      "scope": { /* skill-specific */ },
      "issuedBy": "did:key:z6Mk...",
      "issuedAt": "2026-09-28T10:00:00Z",
      "expiresAt": "2026-09-28T11:00:00Z",
      "signature": "z6Mk.../Base64Signature"
    }
  ],
  "provenance": {
    "aci": "loomwork.dev/example-agent@v1.2.3",
    "aciDigest": "sha256:abc123...",
    "runtime": "loomwork-rs/0.1.0"
  }
}
```

5.4 Capability negotiation handshake

Before any AMP method can be invoked, two agents **MUST** complete the capability negotiation handshake. The handshake establishes a session, declares each side's capabilities and constraints, and issues the initial capability tokens. The handshake is three messages: HELLO, CAPABILITIES, ACCEPT.

AMP Capability Negotiation & Task Delegation

Sequence: a Manager agent discovers a Worker, negotiates capabilities, delegates a subtask, and settles credit.

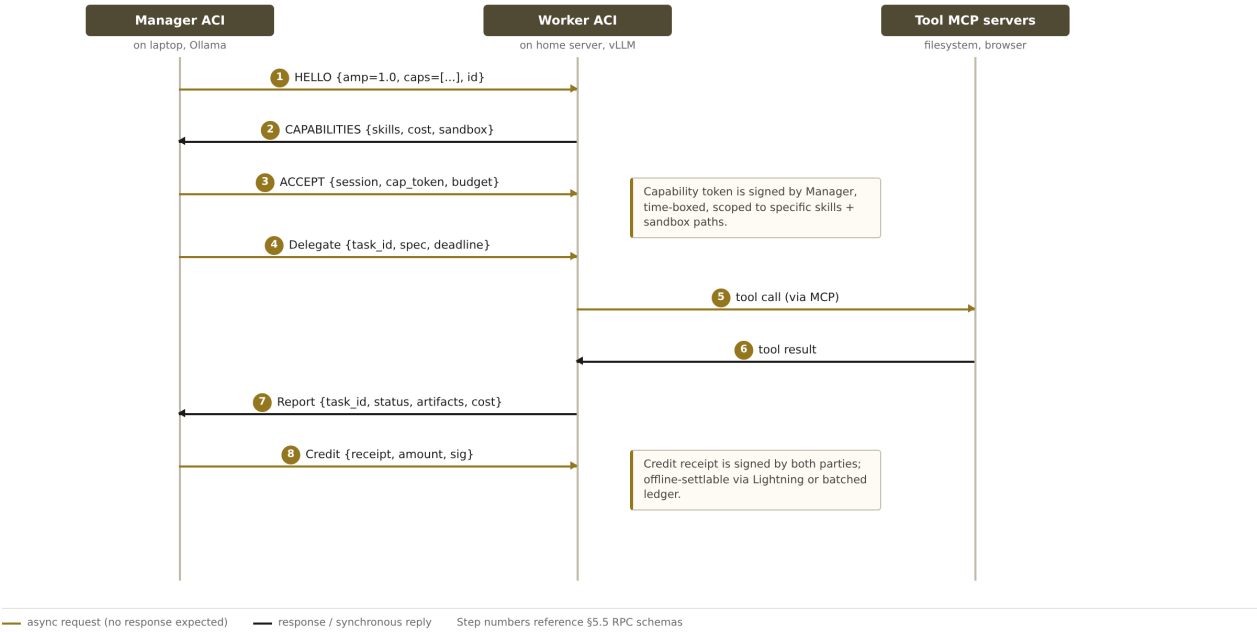


Figure 3. AMP capability negotiation, task delegation, tool call, report, and credit settlement. Step numbers reference §5.5.

```
// HELLO – initiate session
{
  "jsonrpc": "2.0", "id": "req_01H8A...",
  "method": "amp/hello",
  "params": {
    "ampVersion": "0.1",
    "agentId": "did:key:z6Mkmanager...",
    "offeredSkills": ["delegate", "subscribe", "credit"],
    "runtimeInfo": { "name": "loomwork-rs", "version": "0.1.0" },
  },
  "amp": { "version": "0.1", "traceId": "trace_01H8A...", "ttl": 30 },
  "capabilities": [],
  "provenance": { "aci": "loomwork.dev/manager@v1.0.0", ... }
}

// CAPABILITIES – response from peer
{
  "jsonrpc": "2.0", "id": "req_01H8A...", // same id = response
  "result": {
    "sessionId": "sess_01H8B...",
    "acceptedSkills": ["delegate", "subscribe"],
    "declinedSkills": ["credit"], // peer doesn't settle credit
    "offeredSkills": ["research", "summarize"],
    "costModel": { "currency": "loompoint", "per1kTokens": 10 },
    "sandboxPolicy": { "fs.write": ["/tmp/agent/**"] }
  }
}

// ACCEPT – finalize session
{
  "jsonrpc": "2.0", "id": "req_01H8C...",
  "method": "amp/accept",
  "params": {
    "sessionId": "sess_01H8B...",
    "acceptedSkills": ["research", "summarize"],
    "budget": { "maxTokens": 50000, "maxWallSeconds": 300 },
    "capabilityToken": { /* signed token for delegated skills */ }
  }
}
```

5.5 Core RPCs

AMP defines six core RPCs in v0.1. Each RPC has a request shape, a response shape, and a defined error set. The schemas below show the **params** object only; the full envelope (per §5.3) wraps every request and response.

5.5.1 amp/delegate

Delegate a subtask to a peer agent. The peer receives a task spec, a deadline, and a budget; it returns either a task acknowledgment (and later emits **amp/report** messages) or an immediate refusal.

```
// Request: amp/delegate params
{
  "taskId": "task_01H8D...",
  "spec": {
    "intent": "research",
    "inputs": { "topic": "ACI spec adoption" },
    "expectedOutputs": ["summary:string", "citations:array<URL>"],
    "format": "markdown"
  },
  "deadline": "2026-09-28T10:30:00Z",
  "budget": { "maxTokens": 8000, "maxToolCalls": 5 }
}
// Response (acknowledgment)
{ "accepted": true, "estimatedCompletion": "2026-09-28T10:15:00Z" }
// Response (refusal)
{ "accepted": false, "reason": "BUDGET_T00_LOW", "minBudget": { "maxTokens": 12000 }
}
```

5.5.2 amp/negotiate

Negotiate the parameters of a previously-delegated task. Used when the callee realizes the original spec is ambiguous or under-budgeted. Negotiation **MUST** be idempotent within a (taskId, version) pair.

```
// Request: amp/negotiate params
{
  "taskId": "task_01H8D...",
  "version": 2,
  "proposedChanges": {
    "budget.maxTokens": 12000,
    "spec.inputs.depth": "deep"
  },
  "reason": "Insufficient budget for cited research; 12k tokens needed for arxiv search."
}
// Response
{ "accepted": true, "newBudget": { "maxTokens": 12000 }, "effectiveAt":
  "2026-09-28T10:05:00Z" }
```

5.5.3 amp/report

Report progress, partial results, or final results on a delegated task. The callee **MAY** emit multiple amp/report messages before completing; the final report **MUST** set `status = "complete"` or `"failed"`.

```
// Request: amp/report params
{
  "taskId": "task_01H8D...",
  "status": "complete",
  "artifacts": [
    { "name": "summary.md", "mime": "text/markdown", "digest": "sha256:..." },
    { "name": "citations.json", "mime": "application/json", "digest": "sha256:..." }
  ],
  "usage": { "tokensIn": 4200, "tokensOut": 3100, "toolCalls": 4, "wallSeconds": 87 },
  "error": null
}
// Response (ack)
{ "received": true }
```

5.5.4 amp/credit

Settle credit for work performed. The payer (delegator) issues a signed credit receipt; the payee (delegatee) countersigns. Receipts are offline-settlable: they can be accumulated and redeemed later via Lightning, a batched ledger, or a manual process.

```
// Request: amp/credit params
{
  "receiptId": "rct_01H8E...",
  "taskId": "task_01H8D...",
  "amount": { "currency": "loompoint", "value": 73 },
  "breakdown": { "tokens": 50, "toolCalls": 15, "wallSeconds": 8 },
  "payer": "did:key:z6Mkmanager...",
  "payee": "did:key:z6Mkworker...",
  "issuedAt": "2026-09-28T10:16:00Z",
  "payerSig": "z6Mkmanager.../Base64Signature"
}
// Response: payee countersigns
{ "receiptId": "rct_01H8E...", "payeeSig": "z6Mkworker.../Base64Signature" }
```

5.5.5 amp/subscribe and amp/cancel

Subscribe to a stream of events from a peer (e.g. 'notify me when any agent in the mesh completes a research task') and cancel an outstanding task. Both are lightweight: **amp/subscribe** returns a subscription ID that future **amp/report** messages will reference; **amp/cancel** returns the resources consumed so far and ends the task.

5.6 Task graph and shared state

Every delegated task becomes a node in a shared task graph. The graph is a DAG: tasks may spawn subtasks, but cycles are forbidden and the runtime **MUST** detect and reject them. Each node carries its status (pending, running, negotiating, complete, failed, cancelled), its budget consumption, and a reference to its parent. The task graph is the source of truth for mesh-wide observability: any agent with read access to the graph can answer 'what is everyone doing right now?' without polling.

5.7 Credit and accounting

AMP uses an off-ledger credit model: agents exchange signed receipts in real time, and settlement happens out-of-band. This avoids the latency of on-ledger confirmation while preserving auditability. Three settlement mechanisms are anticipated in v0.1: (a) Lightning micropayments for internet-scale peer-to-peer settlement, (b) a batched ledger for organizational deployments where a single operator settles periodically, and (c) manual settlement for trusted-mesh deployments where receipts are logged but not redeemed. The protocol itself is settlement-agnostic: the receipt format is identical across all three.

5.8 Security model

AMP security rests on three pillars: mutual authentication, capability tokens, and replay protection. Mutual authentication uses TLS 1.3 with mutual certificates for WebSocket and libp2p's built-in secio for that transport. Capability tokens are signed by the issuer (the agent granting the capability), scoped to a specific skill, time-boxed, and revocable via an `amp/revoke` message. Replay protection uses a combination of the `amp.ttl` field (messages expire) and a per-session nonce cache (the runtime rejects duplicate nonces within the TTL window).

5.9 Error code registry

AMP errors use JSON-RPC 2.0's error object with a structured `data` field. The error code space is partitioned: 1xx for transport, 2xx for capability, 3xx for task, 4xx for credit, 5xx for protocol.

Code	Name	Meaning	Retry?
100	TRANSPORT_ERROR	Underlying transport failure (WS close, libp2p reset).	Yes, backoff
101	TTL_EXPIRED	Message arrived after <code>amp.ttl</code> window.	No
200	CAPABILITY_MISSING	Sender did not present a required capability token.	No
201	CAPABILITY_EXPIRED	Capability token's <code>expiresAt</code> is in the past.	Yes, after refresh
202	CAPABILITY_REVOKED	Capability was revoked via <code>amp/revoke</code> .	No
300	TASK_NOT_FOUND	Referenced <code>taskId</code> does not exist in the callee's graph.	No
301	TASK_ALREADY_COMPLETE	Callee received a delegate for an already-complete task.	No
302	BUDGET_EXCEEDED	Task budget has been consumed.	Yes, after negotiate
303	DEADLINE_EXCEEDED	Task deadline passed before completion.	No
400	CURRENCY_MISMATCH	Payer and payee use different credit currencies.	No
401	INVALID_RECEIPT	Credit receipt signature failed verification.	No

Code	Name	Meaning	Retry?
402	INSUFFICIENT_BALANCE	Payer's declared balance is below receipt amount.	No
500	PROTOCOL_VIOLATION	Message does not conform to AMP envelope schema.	No
501	VERSION_MISMATCH	amp.version not supported by peer.	No
502	CYCLE_DETECTED	Delegation would create a cycle in task graph.	No

5.10 Version negotiation and backwards compatibility

AMP version negotiation happens in the HELLO message. The initiator declares the highest version it supports; the responder either accepts that version or proposes a lower one in the CAPABILITIES response. If no common version exists, the responder **MUST** return a 501 VERSION_MISMATCH error and the session is not established. Within a major version, newer minor versions **MUST** be backwards-compatible with older ones: a v0.1 runtime **MUST** accept v0.2 messages by ignoring unrecognized optional fields. Breaking changes require a major version bump and explicit negotiation.

6. Relationship to Existing Protocols

Loomwork does not exist in a vacuum. It is the synthesis of patterns that have emerged across a dozen projects over the last 18 months, and it deliberately reuses existing standards wherever they are sufficient. This section maps Loomwork's components to their closest existing analogues and is explicit about what is borrowed and what is new.

Loomwork component	Closest existing analogue	What is borrowed	What is new
ACI	OCI Image Format (Docker)	Archive layout, manifest+digest pattern, Sigstore signing, SLSA provenance	Persona, skills DAG, MCP tool bindings, sandbox policy, memory schema as first-class fields
AMP	MCP (Anthropic)	JSON-RPC 2.0 framing, capability concept, transport-agnostic design	Agent-to-agent (vs agent-to-tool), credit model, task graph, capability tokens
Memory Layer	Mem0 / Zep	Vector + graph hybrid storage, episodic memory concept	Self-hosted by default, E2E encrypted, provenance on every write, follows user across vendors
Vibe-to-Verified	Bolt.new / Totalum	Natural-language to app generation	Mandatory verification gate (SAST, SBOM, formal spec), signed provenance receipt, deploy refusal on failure
Mesh Runtime	Copilot Mission Control	Async task fan-out, parallel agent dispatch, PR-based review	Cross-vendor (works with any ACI), local-first, no central cloud required

The most important borrowing is from OCI. ACI deliberately mirrors the OCI Image Format's manifest+digests+signature pattern because that pattern has been battle-tested at planet scale and because it gives every developer who has ever built a Docker image an immediate mental model for what an ACI is. The most important innovation is AMP. There is no existing open protocol for agent-to-agent delegation, capability negotiation, and credit accounting — MCP explicitly does not cover this, and the alternatives (A2A Protocol, AutoGen's internal protocol) are either proprietary or framework-locked. AMP is the gap-filler.

Honest limitations

Loomwork is not a silver bullet. It does not solve the model-safety problem (that is the model's job), it does not solve the sandbox-escape problem (that is the sandbox implementation's job), and it does not guarantee that agents from different vendors will cooperate smoothly — it only guarantees that they will be able to negotiate whether they can. The specification's value is in making the negotiation possible, not in predetermining its outcome.

7. Roadmap and Risks

This section outlines the planned rollout in three time horizons and enumerates the top risks with mitigations. The roadmap is intentionally concrete: each milestone has a single deliverable that can be independently verified.

7.1 Roadmap

Horizon	Milestone	Deliverable
90 days	v0.1 spec freeze	ACI v0.1 and AMP v0.1 specifications published under CC-BY 4.0; reference Rust runtime; three example ACIs (research, coding, deploy); conformance test suite.
90 days	First demo	End-to-end demo: a Manager ACI on a laptop delegates a research subtask to a Worker ACI on a Raspberry Pi via AMP; both run Ollama locally.
12 months	v1.0 spec	ACI v1.0 (stable manifest), AMP v1.0 (stable wire format); Memory Layer spec; first 50 community ACIs indexed; marketplace alpha.
12 months	Language bindings	Rust (reference), Python, TypeScript, Go bindings; Swift/Android for mobile mesh.
24 months	Full mesh runtime	Cross-device sync, offline-first conflict resolution, Lightning-based credit settlement; 500+ community ACIs; foundation governance.

7.2 Top risks and mitigations

#	Risk	Likelihood	Impact	Mitigation
R 1	Anthropic (or another major vendor) closes the protocol layer with a proprietary solution before v1.0	Medium	Existential	Ship v0.1 within 90 days; aggressively recruit early adopters; pursue standardization under OpenSSF / Linux Foundation
R 2	Open-source community fragments into competing standards	Medium	High	Be aggressively neutral on runtime, model, and IDE; explicitly invite competitors to join working group; reject vendor-specific extensions in core spec
R 3	Security hole in agent sandbox leads to a high-profile escape	Medium	High	Mandate SLSA Level 3 for all reference ACIs; ship conformance tests that include negative-path sandbox tests; publish a security disclosure policy with 90-day window
R 4	UX complexity (5 layers, 6 RPCs, multiple transports) deters casual users	High	Medium	Ship a 'batteries-included' default runtime that hides complexity; expose advanced controls only when user opts in
R 5	Cold-start problem: marketplace has no agents, so no users; no users, so no agents	High	High	Ship 3 reference ACIs on day 1; sponsor a community ACI hackathon; reach out to OpenHands/OpenClaw maintainers about repackaging as ACIs

7.3 Open questions

The following questions are explicitly unresolved as of v0.1 and are flagged for community discussion during the 90-day review window.

Q1. Should the ACI manifest support multiple personas in one archive (e.g. an agent that presents different behavior in different contexts), or is one ACI one persona?

Q2. How should AMP handle agents that are offline at delegation time — fire-and-forget queues, or strict refusal?

Q3. What is the right governance model for the spec — single-vendor stewardship, working group under a foundation, or a fully decentralized process?

Q4. Should the Memory Layer be part of the ACI spec, a separate spec, or left to runtime implementations entirely?

Q5. How should AMP handle delegation across trust boundaries (e.g. agent on personal laptop delegates to agent on a public cloud)? Is mutual TLS enough, or is a third-party attestation required?

Q6. Should ACI include a 'kill switch' field that allows the issuer to remotely terminate a running instance of their agent? What are the implications for trust?

Q7. What is the minimum viable subset of AMP that a runtime must implement to call itself 'AMP-compatible'? The current spec requires all six RPCs, which may be too high a bar.

Q8. How does Loomwork relate to emerging standards like A2A Protocol and ACP? Should it merge, compete, or explicitly defer to one of them?

Appendix A. Glossary

Term	Definition
ACI	Agent Container Image. The signed, portable package format for an AI agent defined in §4. An ACI bundles persona, skills, tool bindings, sandbox policy, memory schema, and provenance into a single loadable archive.
AMP	Agent Mesh Protocol. The peer-to-peer wire protocol for agent-to-agent communication defined in §5. AMP is layered on top of MCP at the conceptual level and uses independent transports.
MCP	Model Context Protocol. Anthropic's open protocol for connecting AI agents to tools and data sources, introduced in 2024. MCP standardizes agent-to-tool communication; AMP standardizes agent-to-agent.
OCI	Open Container Initiative. The body that standardizes container formats and runtimes, including the OCI Image Format that ACI's archive layout deliberately mirrors.
SLSA	Supply-chain Levels for Software Artifacts. A framework for hardening software supply chains, expressed as a series of levels. ACI requires SLSA Level 3 or higher for provenance attestations.
Sigstore	An open-source software signing stack using ephemeral keys and a transparency log. ACI uses cosign (part of Sigstore) for manifest signatures and Fulcio (Sigstore's CA) for signing certificates.
libp2p	A modular network stack originally developed for IPFS, providing peer routing, content discovery, and encrypted transport. One of AMP's three transport bindings.
JSON-RPC 2.0	A stateless, light-weight remote procedure call protocol encoded in JSON. AMP's message envelope is JSON-RPC 2.0 with three AMP-specific extensions.
Capability token	A signed, time-boxed, scope-limited token issued by one agent to another, granting the recipient the right to invoke a specific skill. AMP capability tokens are revocable.
Credit receipt	A signed record of work performed by one agent for another, used for offline settlement. AMP credit receipts are dual-signed (payer and payee) and can be redeemed via Lightning, a batched ledger, or manually.
Agent mesh	A peer-to-peer network of agents that communicate via AMP. A mesh has no central broker; any two compliant runtimes can converse directly.
Vibe-to-Verified	Loomwork's end-to-end pipeline from natural-language spec to deployable, verified application. Includes SAST, SBOM, optional formal-spec checking, and signed provenance attestation. The 'verified' in the name refers to the mandatory verification gate.
Sandbox	The isolated execution environment in which an ACI runs. The sandbox enforces the filesystem, network, and syscall constraints declared in sandbox.json. Implementations may use Docker, Firecracker, gVisor, or native seccomp.
Provenance	A signed attestation linking an artifact (an ACI, an agent action, a memory write) to its source. Provenance is the foundation of Loomwork's auditability guarantee.